

Theater Ticketing System

Software Requirements Specification

V4

03/05/2026

Group 7

Alvin Hoang, Brandon Tse, David Nguyen

Prepared for
CS 250 - Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Spring 2026

Revision History

Date	Description	Author	Comments
2/12/26	Version 1	David Nguyen	First Revision
2/12/26	Version 1	Alvin Hoang	First Revision
2/12/26	Version 1	Brandon Tse	First Revision
2/19/26	Version 2	David Nguyen	Second Revision
2/19/26	Version 2	Alvin Hoang	Second Revision
2/19/26	Version 2	Brandon Tse	Second Revision
2/19/26	Version 3	David Nguyen	Third Revision
3/5/26	Version 4	David Nguyen	Second Revision
3/5/26	Version 4	Alvin Hoang	Second Revision
3/5/26	Version 4	Brandon Tse	Second Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

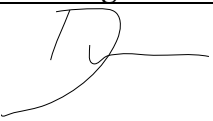
Signature	Printed Name	Title	Date
	David Nguyen	Software Engineer	02/17/26
Brandon Tse	Brandon Tse	Software Engineer	02/17/26
Alvin Hoang	Alvin Hoang	Software Engineer	02/17/26
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History.....	2
Document Approval.....	2
1. Introduction.....	1
1.1 Purpose.....	1
1.2 Scope.....	1
1.3 Definitions, Acronyms, and Abbreviations.....	2
1.4 References.....	2
1.5 Overview.....	2
2. General Description.....	3
2.1 Product Perspective.....	3
2.3 User Characteristics.....	3
2.4 General Constraints.....	4
2.5 Assumptions and Dependencies.....	4
3. Specific Requirements.....	5
3.1 External Interface Requirements.....	5
3.1.1 User Interfaces.....	5
3.1.2 Hardware Interfaces.....	5
3.1.3 Software Interfaces.....	5
3.1.4 Communications Interfaces.....	5
3.2 Functional Requirements.....	5
3.2.1 Process Ticket Payment.....	5
3.2.1.1 Introduction.....	5
3.2.1.2 Inputs.....	6
3.2.1.3 Processing.....	6
3.2.1.4 Outputs.....	6
3.2.1.5 Error Handling.....	6
3.2.2 Payment Authorization Validation.....	6
3.2.2.1 Introduction.....	6
3.2.2.2 Inputs.....	6
3.2.2.3 Processing.....	7
3.2.2.4 Outputs.....	7
3.2.2.5 Error Handling.....	7
3.2.3.1 Seat Reservation.....	7
3.2.3.2 Inputs.....	7
3.2.3.3 Processing.....	7
3.2.3.4 Outputs.....	7
3.2.3.5 Error Handling.....	8
3.2.4 Ticket Purchase Restrictions.....	8
3.2.4.1 Processing.....	8

Theater Ticketing System

3.2.4.2 Error Handling.....	8
3.2.5 Bot Prevention.....	8
3.2.5.1 Processing.....	8
3.2.6 Discount Processing.....	8
3.2.6.1 Processing.....	8
3.2.7 Administrator Management.....	9
3.2.7.1 Processing.....	9
3.3 Use Cases.....	9
3.3.1 Complete Ticket Payment.....	9
3.3.2 Retry Payment.....	10
3.3.3 Reserve Seat.....	10
3.4 Classes / Objects.....	11
3.4.1 Payment.....	12
3.4.2 Transaction.....	12
3.4.3 Reservation.....	13
3.4.4 Movie.....	13
3.4.5 Showtime.....	14
3.5 Non-Functional Requirements.....	14
3.5.1 Performance.....	14
3.5.2 Reliability.....	15
3.5.3 Availability.....	15
3.5.4 Security.....	15
3.5.5 Maintainability.....	15
3.5.6 Portability.....	15
3.6 Inverse Requirements.....	15
3.7 Design Constraints.....	15
3.8 Logical Database Requirements.....	16
3.9 Other Requirements.....	16
4. Analysis Models.....	16
4.1 Sequence Diagrams.....	17
4.3 Data Flow Diagrams (DFD).....	17
4.2 State-Transition Diagrams (STD).....	18
5. Change Management Process.....	19
A. Appendices.....	19
A.1 Appendix 1.....	20
A.2 Appendix 2.....	20
6. Software Design Specification (Design Document).....	20
6.1 System Description (Developer Overview).....	20
6.2 Software Architecture Overview.....	20
6.3 UML Class Diagram (Design-Level).....	22
6.4 Class Specifications (Concise but “Rubric-Level” Detailed).....	23

6.5 Development Plan and Timeline.....	25
Task 1 - Project Setup and CI.....	25
Task 2 - Database Schema and ORM.....	25
Task 3 - Reservation and Seat Locking.....	26
Task 4 - Checkout and Payment Integration.....	26
Task 5 - Ticket Generation and Email.....	26
Task 6 - Administrative Features.....	26
Task 7 - Testing and System Hardening.....	27
6.5.2 Team Member Responsibilities.....	27
Alvin Hoang.....	27
Brandon Tse.....	27
David Nguyen.....	27
6.4.5 User.....	27
6.4.6 Movie.....	28
6.4.7 Theater.....	28
6.4.8 Showtime.....	29
6.4.9 Seat.....	29
6.4.10 Discount.....	30
6.4.11 PaymentGatewayAdapter.....	30
6.4.12 EmailServiceAdapter.....	31
6.4.13 CaptchaAdapter.....	31
6.4.14 PaymentDetails.....	31
6.4.15 GatewayResult.....	31
6.4.16 ReservationService (Design Notes).....	32
6.4.17 PaymentService.....	32
6.4.18 TicketService (Design Notes).....	32
6.5.3 Project Timeline (17 Week Schedule).....	33
Weeks 1-3.....	33
Weeks 4-6.....	33
Weeks 7-8.....	33
Weeks 9-11.....	33
Weeks 12-13.....	33
Weeks 14-15.....	33
Weeks 16-17.....	34

1. Introduction

The Theater Ticketing System is a web-based movie ticketing platform that is designed to support both online customers and in-person theater employees. In addition, it will also maintain synchronized, real-time ticket availability across all theaters. This Software Requirements Specification defines the functional and non-functional requirements necessary to design, implement, test, and deploy the system successfully.

The system will serve 20 theaters located in the San Diego area and operate within the Pacific Time Zone. It will replace the current slow and error-prone ticketing system with a more reliable, scalable, and secure solution.

1.1 Purpose

The purpose of this SRS is to describe the functional and non-functional requirements of the Theater Ticketing System.

This document provides a detailed description of the system's capabilities, interfaces, constraints, and expectations. It is supposed to be a reference for developers, project managers, stakeholders, and testers involved in the design, implementation, and maintenance of the system.

1.2 Scope

The Theater Ticketing System is a web-based application designed to allow customers to look at movies, select showtimes, reserve seats, and purchase tickets online.

The system will:

- Provide real-time seat availability
- Enable secure online payments
- Generate barcodes and QR codes for each respective ticket
- Send a confirmation email
- Provide tools for the owners so that they're able to manage each movie, its pricing, and schedules.
- Support ticket purchases up to two weeks in advance
- Support assigned seating for deluxe theaters and general admission for regular theaters
- Allow discount pricing for students, military personnel, and seniors

The system will not:

- Process refunds online
- Sell concessions
- Provide streaming services

Theater Ticketing System

The goal of the system is to streamline ticket purchasing, reduce wait times for purchasing tickets in person, and to also improve ease of changing movie parameters such as schedules and pricing.

1.3 Definitions, Acronyms, and Abbreviations

This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.

This document follows standard IEEE SRS formatting conventions:

- “Shall” indicates a mandatory requirement.
- “Should” indicates a recommended requirement.
- “May” indicates an optional feature.
- Numbered sections are hierarchical and organized by feature category.
- Functional requirements are grouped under Section 3.2 and follow a structured format including inputs, processing, outputs, and error handling.

1.4 References

PCI Security Standards Council. (2024). *Payment Card Industry Data Security Standard: Requirements and Testing Procedures (Version 4.0.1)*. PCI Security Standards Council. https://docs-prv.pcisecuritystandards.org/PCI%20DSS/Standard/PCI-DSS-v4_0_1.pdf

1.5 Overview

The remaining sections of this SRS provide a general description, including characteristics of the users of this project, the product's hardware, and the functional and data requirements of the product. The general description of the project is discussed in section 2 of this document. Section 3 gives the functional requirements, data requirements and constraints and assumptions made while designing the Ticketing System. It also gives the user viewpoint of the product. Section 3 also gives the specific requirements of the product. Section 3 also discusses the external interface requirements and gives detailed description of functional requirements. Section 4 is for supporting information.

The remainder of this SRS is organized as follows:

- Section 2 provides a general description of the system, including user characteristics, constraints, and assumptions.
- Section 3 describes the specific functional and non-functional requirements of the system.
- Section 4 presents analysis models such as sequence diagrams, data flow diagrams, and state-transition diagrams.
- Section 5 describes the change management process for maintaining and updating system requirements.

- The appendices contain supporting documentation and additional reference material.

2. General Description

This section of the SRS should describe the general factors that affect 'the product and its requirements'. It should be made clear that this section does not state specific requirements; it only makes those requirements easier to understand.

2.1 Product Perspective

The Theater Ticketing System is a web based system that uses other services such as payment systems as well as email services.

It will interact with: A database for data storage, a payment gateway so that people can pay with online services, an email system so that the ticketing system can send email confirmation, and other hardware such as ticket printers and barcode scanners.

2.2 Product Functions

The Major functions of the System will be:

- Account registration + login
- Movie Searching
- Time selection for each movie
- Seat selection with a chart of the theater
- Ticket reservation
- Cart for tickets
- Online payment processing
- Generation of tickets with ID and QR code
- Email confirmation
- Allow Owner to manage movies, pricing, and schedule

2.3 User Characteristics

Customers:

- Audience that will purchase tickets
- Basic knowledge of the internet
- Access to the ticketing system with either a web browser on a mobile or desktop device

Owner:

- Theater management or other staff
- Manage movies, schedules, and pricing
- Need secure login information
- Moderate knowledge of the internet

System Admin:

- Responsible for maintenance of system

Theater Ticketing System

- Manage database
- Advanced knowledge of the internet

2.4 General Constraints

- The system will use HTTPS for communications
- Payment processing will comply with PCI-DSS
- It will need a payment system such as Stripe
- The database must be relational
- It will comply with privacy regulations
- Passwords will be stored using secure methods
- The system shall be compatible with major web browsers (Chrome, Edge, Safari, Firefox).
- The system shall operate within standard internet bandwidth conditions.
- The system shall not require specialized hardware for customers.
- Administrative users shall require secure authentication credentials.

2.5 Assumptions and Dependencies

Assumptions:

- The theater will have reliable internet connection so that people can buy tickets online
- Users have access to a web browser and will be able to connect to the internet
- The payment service will be operational
- The email provider will be able to reliably email ticket information to customers
- Theater staff will have the ability to use the administration tools
- The seating layout will be predetermined and will be consistent
- The information that the customer provides about their payment information will be correct
- The system will be hosted in an environment that has adequate capacity
- It is assumed that users have access to reliable internet connectivity.
- It is assumed that third-party services (payment gateway and email provider) remain operational.
- It is assumed that the hosting environment supports scalable concurrent user traffic.

Dependencies:

- Payment system: a third party payment system to handle transactions securely
- Email provider: external email service to send ticket confirmation
- Database: needs a database in order to store movie information, schedules, seat availability, user data, and transaction history
- Barcode/Qr code library: needs a system to generate barcodes and QR codes
- Hardware: ticket printers and/or barcode scanners for digital tickets.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- The system shall provide a secure web-based payment interface accessible via modern browsers (Chrome, Edge, Safari, Firefox).
- The interface shall display:
 - Reservation ID
 - Total payment amount
 - Payment method options (credit/debit card, supported digital wallets)
- The system shall display a confirmation message after payment approval or a failure message if payment is declined.

3.1.2 Hardware Interfaces

- The system shall operate on standard user devices (desktop, tablet, mobile).
- No specialized hardware shall be required for payment processing.

3.1.3 Software Interfaces

- The system shall integrate with a third-party payment gateway (e.g., Stripe, PayPal).
- The system shall interface with the reservation database to verify active reservations before processing payment.
- The system shall integrate with an email service to send payment confirmations.

3.1.4 Communications Interfaces

- All communication between the user and system shall use HTTPS.
- Payment transactions shall use encrypted API communication with the payment gateway.

3.2 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.2.1 Process Ticket Payment

3.2.1.1 Introduction

This function processes customer payments for reserved movie tickets.

3.2.1.2 Inputs

- Reservation ID
- Total payment amount
- Card number or digital wallet credentials
- Expiration date
- CVV
- Billing address

3.2.1.3 Processing

- Validate required input fields.
- Verify reservation ID is valid and active.
- Confirm payment amount matches reservation total.
- Submit transaction request to payment gateway.
- Receive transaction approval or denial.
- Store transaction record in database.

3.2.1.4 Outputs

- Payment confirmation message (success or failure).
- Transaction ID (if successful).
- Electronic receipt sent via email.

3.2.1.5 Error Handling

- If required fields are missing, display error message.
- If reservation ID is invalid, cancel transaction and notify user.
- If payment is declined, notify user and allow retry.
- If gateway communication fails, log error and prompt retry.

3.2.2 Payment Authorization Validation

3.2.2.1 Introduction

This function validates user payment information and authorizes the transaction before final processing.

3.2.2.2 Inputs

- Card number or digital wallet credentials
- Expiration date
- CVV
- Billing address

Theater Ticketing System

- Reservation ID
- Total amount

3.2.2.3 Processing

- Validate input format (card length, expiration date format, required fields).
- Verify billing information is complete.
- Confirm reservation ID is valid and active.
- Send authorization request to payment gateway.
- Receive authorization approval or denial.

3.2.2.4 Outputs

- Authorization approval or denial message.
- Authorization code (if approved).

3.2.2.5 Error Handling

- Display error if required fields are missing.
- Display error if card format is invalid.
- Display message if authorization is declined.
- Log failed authorization attempts for security review.

3.2.3.1 Seat Reservation

This function allows customers to temporarily reserve selected seats for a specific showtime.

3.2.3.2 Inputs

- Movie ID
- Showtime ID
- Selected seat numbers

3.2.3.3 Processing

- Verify selected seats are available.
- Lock seats for a maximum of 10 minutes.
- Generate Reservation ID.
- Update database to reflect temporary hold.

3.2.3.4 Outputs

- Reservation ID
- Confirmation message

3.2.3.5 Error Handling

- If seats are unavailable, display error message.
- If session expires, release seats automatically.

3.2.4 Ticket Purchase Restrictions

This function will restrict customers from abusing the ticket system and balance the system to make it fair.

3.2.4.1 Processing

- Customers shall not purchase more than 20 tickets per transaction.
- Tickets shall only be available 14 days prior to showtime.
- Ticket purchases shall be disabled 10 minutes after showtime begins.

3.2.4.2 Error Handling

- Display a restriction message if limits are exceeded.

3.2.5 Bot Prevention

This function will prevent spam from occurring in the system and abuse of bots to get tickets.

3.2.5.1 Processing

- The system shall implement CAPTCHA verification at checkout.
- The system shall detect rapid repeated purchase attempts.
- Suspicious IP addresses shall be temporarily blocked.

3.2.6 Discount Processing

3.2.6.1 Processing

- The system shall verify discount eligibility category.
- The system shall apply discount percentage automatically.
- The system shall log discount usage.

3.2.7 Administrator Management

3.2.7.1 Processing

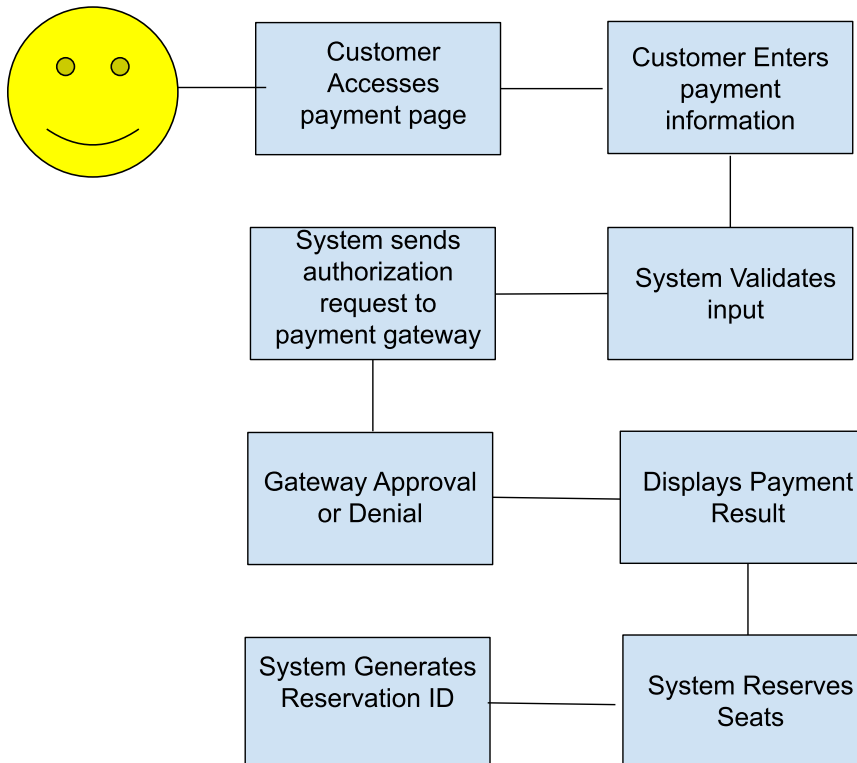
- Add, update, or remove movies.
- Modify ticket pricing.
- Configure showtimes.
- Update seating layout.

3.3 Use Cases

3.3.1 Complete Ticket Payment

Actor: Customer

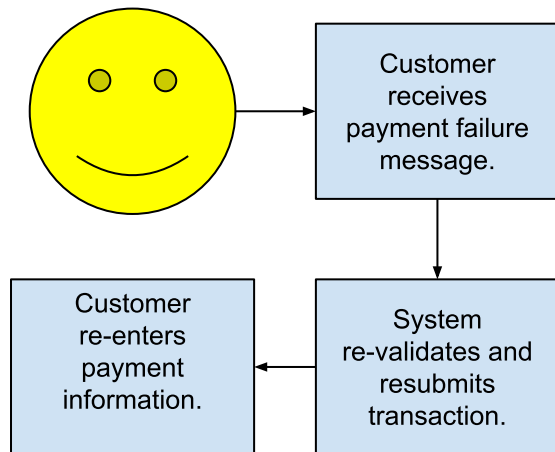
1. Customer accesses payment page.
2. Customer enters payment information.
3. System validates input.
4. System sends authorization request to payment gateway.
5. Gateway returns approval or denial.
6. System displays payment result.
7. System stores transaction record and sends receipt (if approved).



3.3.2 Retry Payment

Actor: Customer

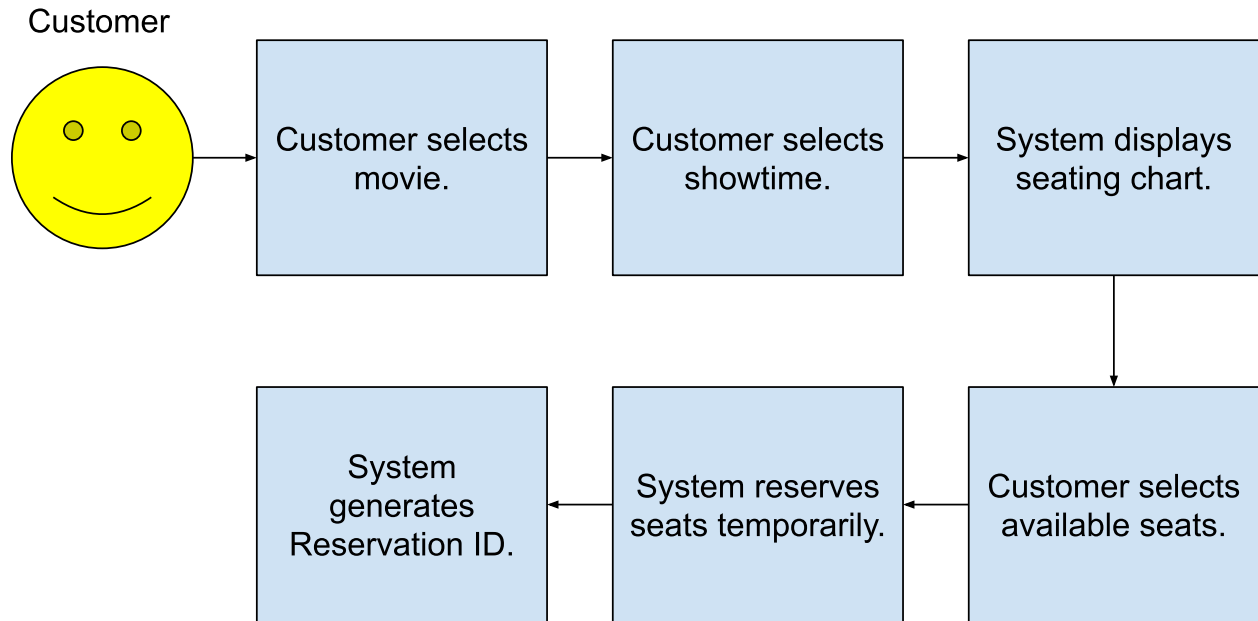
1. Customer receives payment failure message.
2. Customer re-enters payment information.
3. System re-validates and resubmits transaction.



3.3.3 Reserve Seat

1. Customer selects movie.
2. Customer selects showtime.
3. System displays seating chart.
4. Customer selects available seats.
5. System reserves seats temporarily
6. System generates Reservation ID.

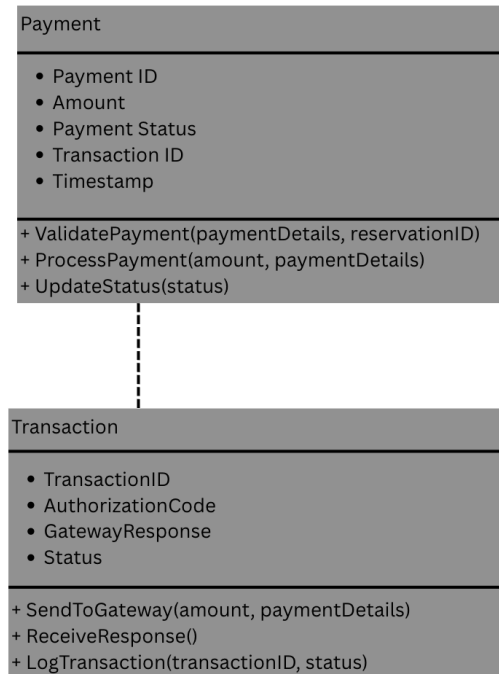
Theater Ticketing System



3.4 Classes / Objects

UML Diagram for Payment and Transaction Processes

Theater Ticketing System



3.4.1 Payment

3.4.1.1 Attributes

- PaymentID
- ReservationID
- Amount
- PaymentStatus
- TransactionID
- Timestamp

3.4.1.2 Functions

- ValidatePayment()
- ProcessPayment()
- UpdateStatus()

(References: 3.2.1, 3.2.2)

3.4.2 Transaction

3.4.2.1 Attributes

- TransactionID

Theater Ticketing System

- AuthorizationCode
- GatewayResponse
- Status

3.4.2.2 Functions

- SendToGateway()
- ReceiveResponse()
- LogTransaction()

(References: 3.2.1, 3.2.2)

3.4.3 Reservation

3.4.3.1 Attributes

- ReservationID
- ShowtimeID
- SeatNumbers
- Status (Pending, Confirmed, Expired)
- Timestamp

3.4.3.2 Functions

- LockSeats()
- ReleaseSeats()
- ConfirmReservation()

3.4.3.3 States

- Pending
- Confirmed
- Expired

3.4.3.4 Relationships

- Associated with Payment
- Associated with Showtime

3.4.4 Movie

3.4.4.1 Attributes

- MovieID

Theater Ticketing System

- Title
- Description
- Duration
- Rating

3.4.4.2 Functions

- AddMovie()
- UpdateMovie()
- RemoveMovie()

3.4.5 Showtime

3.4.5.1 Attributes

- ShowtimeID
- MovieID
- TheaterID
- DateTime
- AvailableSeats

3.4.5.2 Functions

- UpdateAvailability()
- CloseShowtime()

3.5 Non-Functional Requirements

Non-functional requirements may exist for the following attributes. Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transaction shall be processed in less than a second, system downtime may not exceed 1 minute per day, > 30 day MTBF value, etc).

3.5.1 Performance

- 95% of payment transactions shall be completed within 3 seconds.
- Authorization responses shall be processed within 2 seconds under normal operating conditions.
- The system shall support at least 1000 concurrent users without performance degradation.
- Seat reservation conflicts shall not exceed 2% under peak load.

3.5.2 Reliability

- The system shall maintain a 99% successful transaction processing rate, excluding user-declined payments.
- Failed transactions shall be logged for audit purposes.

3.5.3 Availability

- The payment processing module shall maintain 99% uptime per month.

3.5.4 Security

- All payment data shall be encrypted in transit using TLS 1.2 or higher.
- The system shall comply with PCI-DSS standards.
- The system shall not store full credit card numbers.
- The system shall not store CVV codes after transaction completion.
- The system shall implement CAPTCHA validation during checkout.
- Repeated failed transactions from the same IP address shall trigger temporary account or IP restriction.

3.5.5 Maintainability

- The payment gateway integration shall be modular to allow replacement without affecting other system components.
- System logs shall be maintained to assist troubleshooting and auditing.

3.5.6 Portability

- The payment processing module shall function on major web browsers.
- The system shall be accessible on Windows, macOS, Android, and iOS devices via browser.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitation, etc. that will impact this software project.

- *Security constraints*
 - *Payment must be handled by a third-party gateway.*
- *Time constraints*

Theater Ticketing System

- *Seat reservations will expire after 10 minutes if purchase isn't finished*
 - *Tickets become available 14 days before showtime*
- *Platform Constraints*
 - *Must run as a web-based system accessible by common web browsers*
- *Anti-bot-constraints*
 - *Must include CAPTCHA*

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

A relational database will be used to store users, movies, showtimes, seats, reservations, payments, transactions, discounts, and tickets

- *Identifiers*
 - *Use UUID for primary values such as Movie, showtime, reservation, payment, transaction, ticket*
- *Money*
 - *Store currency values as amountCents: BIGINT*
- *Seat Representation*
 - *Seat numbers should be stored as strings such as A10 or combined as row:CHAR + seat: INT*
- *To prevent seats being double booked*
 - *Unique constraints should be used such as (showtime_id, seat_number for confirmed seats*
 - *Reservation expiration will automatically release seats*
- *Retention*
 - *Transaction records are retained for accounting and potential manual refund processing outside the system*
 - *Store only the payment gateway and not the actual card information*

3.9 Other Requirements

Catchall section for any additional requirements.

- *Email confirmation*
 - *After a successful purchase, system will send a confirmation email and receipt*
- *Usability*
 - *Seat selection UI must show real-time availability as well as showing which seats are already taken clearly*
- *Logging/monitoring*
 - *Log checkout failures, suspicious activity, payment gateway errors and reservation expirations.*

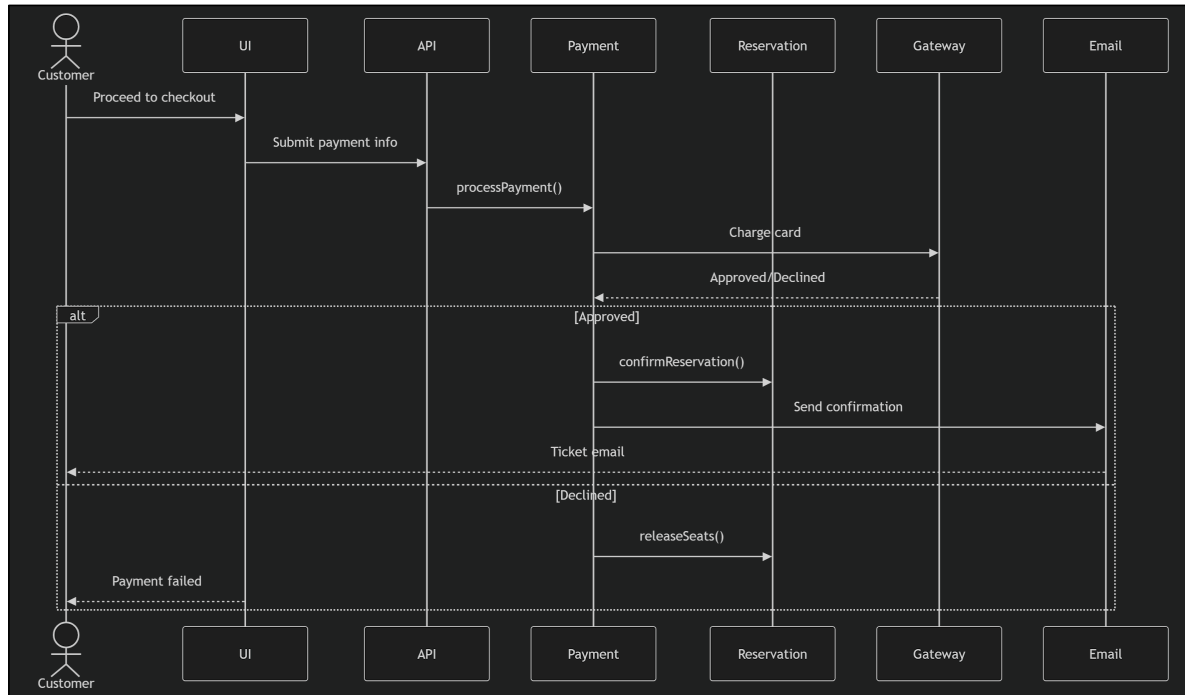
4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable the SRS's requirements.

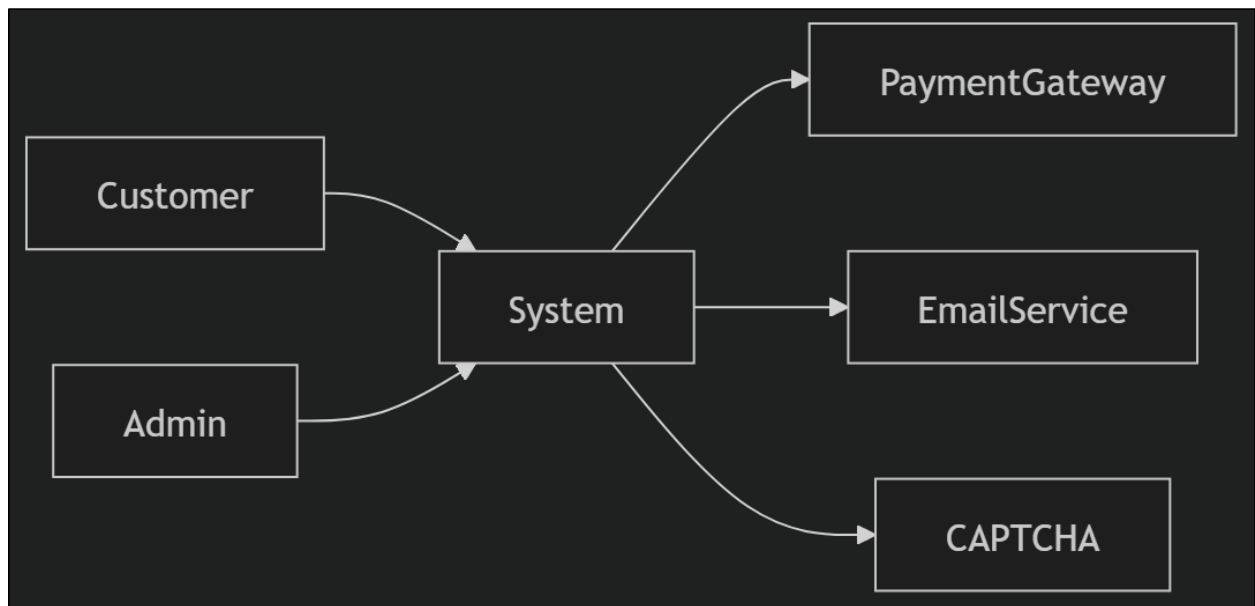
Theater Ticketing System

4.1 Sequence Diagrams

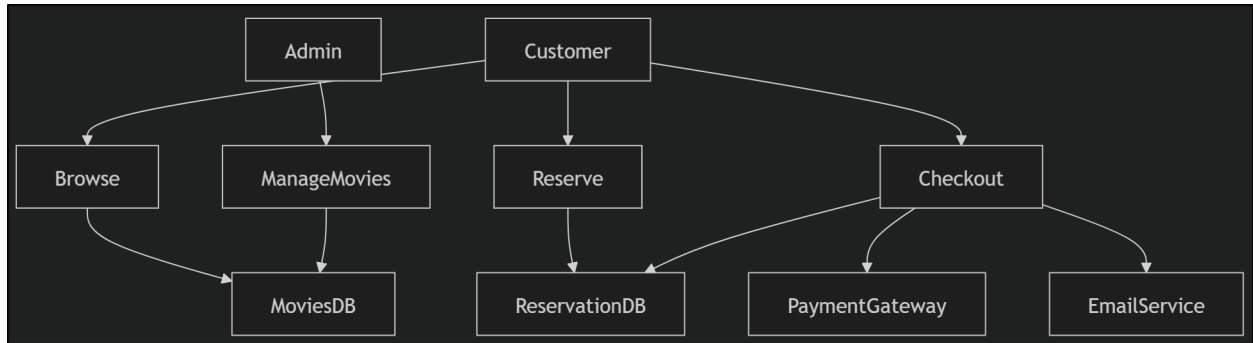
- Figure 1: Seat Reservation



4.3 Data Flow Diagrams (DFD)

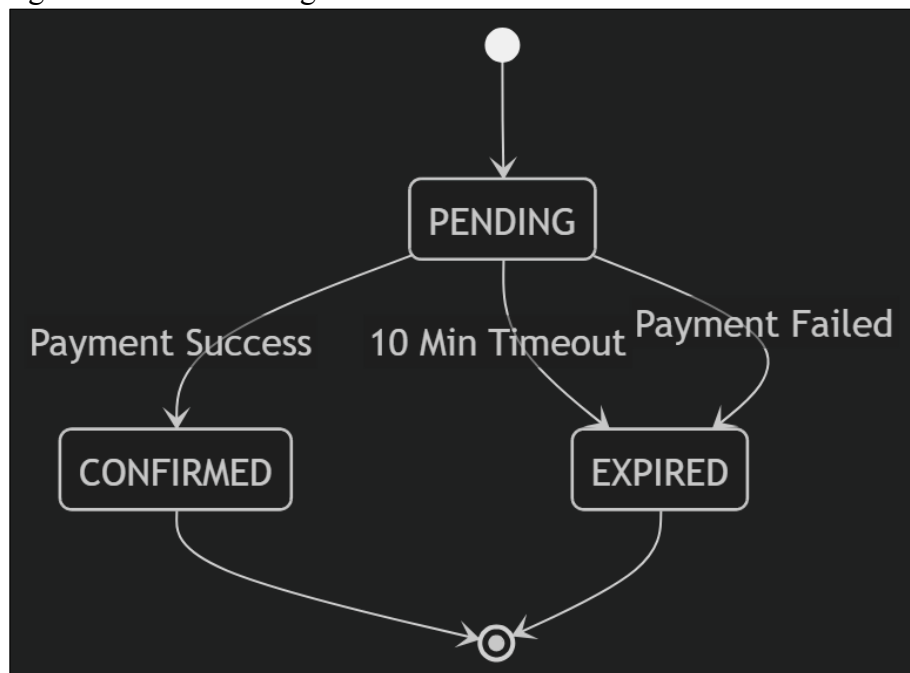


Theater Ticketing System

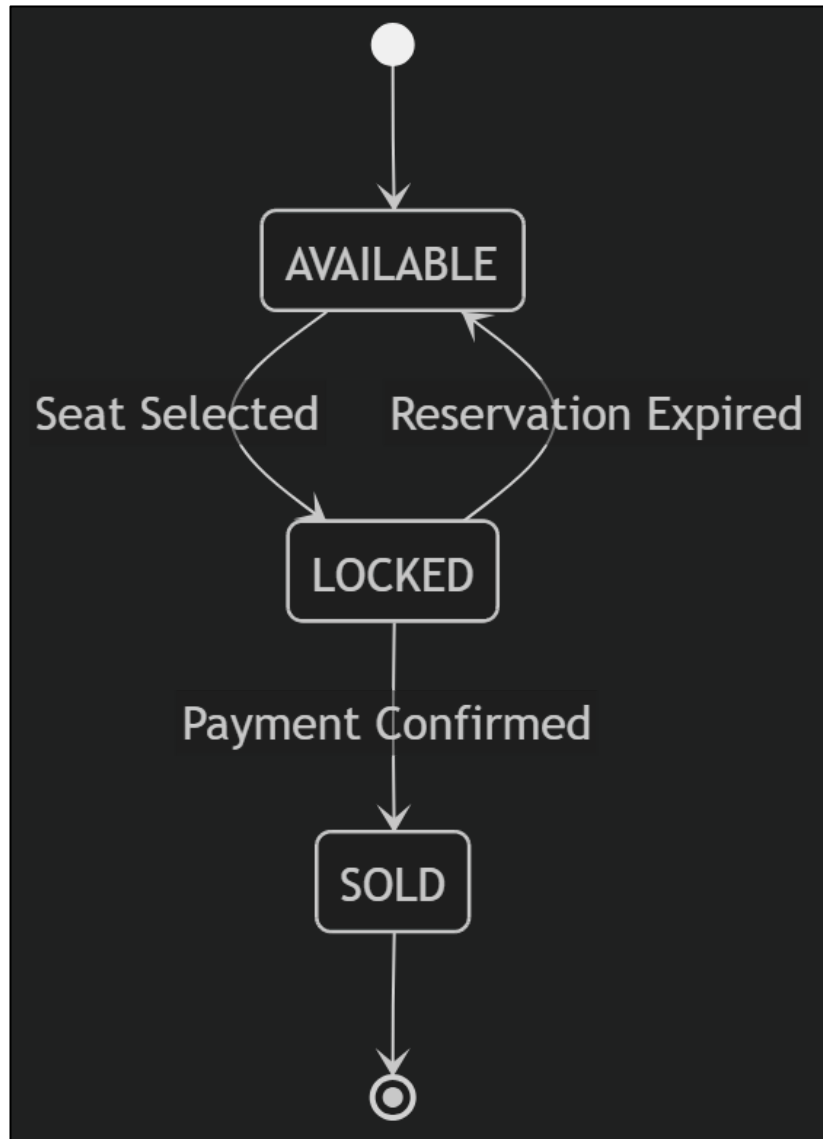


4.2 State-Transition Diagrams (STD)

Figure Reservation Diagram



Seat Diagram



5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2

6. Software Design Specification (Design Document)

6.1 System Description (Developer Overview)

The Theater Ticketing System is a web-based application supporting customers and theater staff for browsing movies, selecting showtimes, reserving seats, purchasing tickets, and receiving electronic tickets (QR/barcode). Administrative users manage movies, showtimes, pricing, and seating layouts. The system integrates with third-party services for payment processing, email delivery, and bot protection (CAPTCHA).

6.2 Software Architecture Overview

6.2.1 Architectural Style

The system uses a **3-tier architecture**:

1. **Presentation Layer (Web UI)**: Customer/admin web pages for browsing, reservation, checkout, and administration.
2. **Application Layer (REST API)**: Business logic services for reservations, payments, ticket generation, and admin management.
3. **Data Layer (Relational DB)**: Stores movies, showtimes, seats, reservations, payments, tickets, and audit logs.

External dependencies are accessed via service adapters (Payment Gateway, Email Service, CAPTCHA).

6.2.2 Major Components and Responsibilities

- **Web UI**
 - Customer: browse, select seats, checkout
 - Admin: manage movies/showtimes/pricing/seating layouts
- **API Gateway / Controllers**
 - Validates requests, invokes services, returns responses

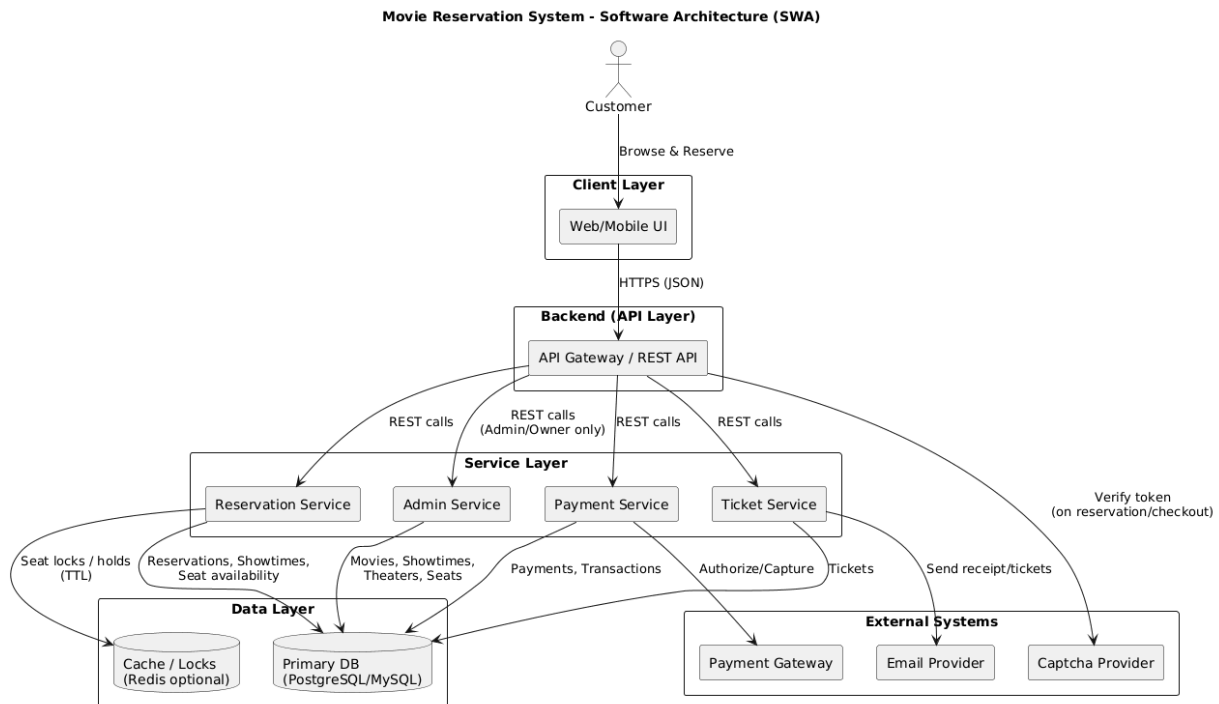
Theater Ticketing System

- **Reservation Service**
 - Seat locking, reservation creation/expiration, conflict checks
- **Payment Service**
 - Payment authorization + charge, transaction recording, receipts
- **Ticket Service**
 - Generates ticket records and QR/barcode payloads
- **Admin Service**
 - CRUD for Movie/Showtime/Pricing/Layout
- **Database**
 - Persist all entities with integrity constraints
- **External Services**
 - Payment Gateway (Stripe/PayPal)
 - Email Provider
 - CAPTCHA Provider

6.2.3 Architecture Diagram (structure)

Theater Ticketing System Architecture Diagram

Theater Ticketing System



This software architecture diagram shows the overall structure of the movie reservation system and how all the main parts connect to each other. A customer uses the website or mobile app (the UI) to browse movies, pick a showtime, select seats, and check out. The UI sends requests over the internet (using HTTPS) to the backend API, which acts as the main entry point to the system. The API receives requests, checks that they are valid and that the user has permission, and then sends them to the correct internal service.

Behind the API are several services that each handle a specific responsibility. The Reservation Service handles seat selection and booking. The Payment Service processes payments and talks to an external payment provider to charge the customer. The Ticket Service creates tickets after payment is successful and sends them to the customer by email. The Admin Service allows staff to add movies, create showtimes, and manage theater information. Each service focuses on one job to keep the system organized and easier to maintain.

All of these services store and read information from the main database, which keeps records of users, movies, showtimes, seats, reservations, payments, and tickets. The system may also use a temporary locking system to hold seats for a short time so that two people cannot book the same seat at once. In addition, the system connects to outside services: a payment gateway to process credit cards, an email provider to send tickets and receipts, and a captcha service to prevent bots. Together, these parts show the complete structure of the system and how everything works together to support online movie reservations.

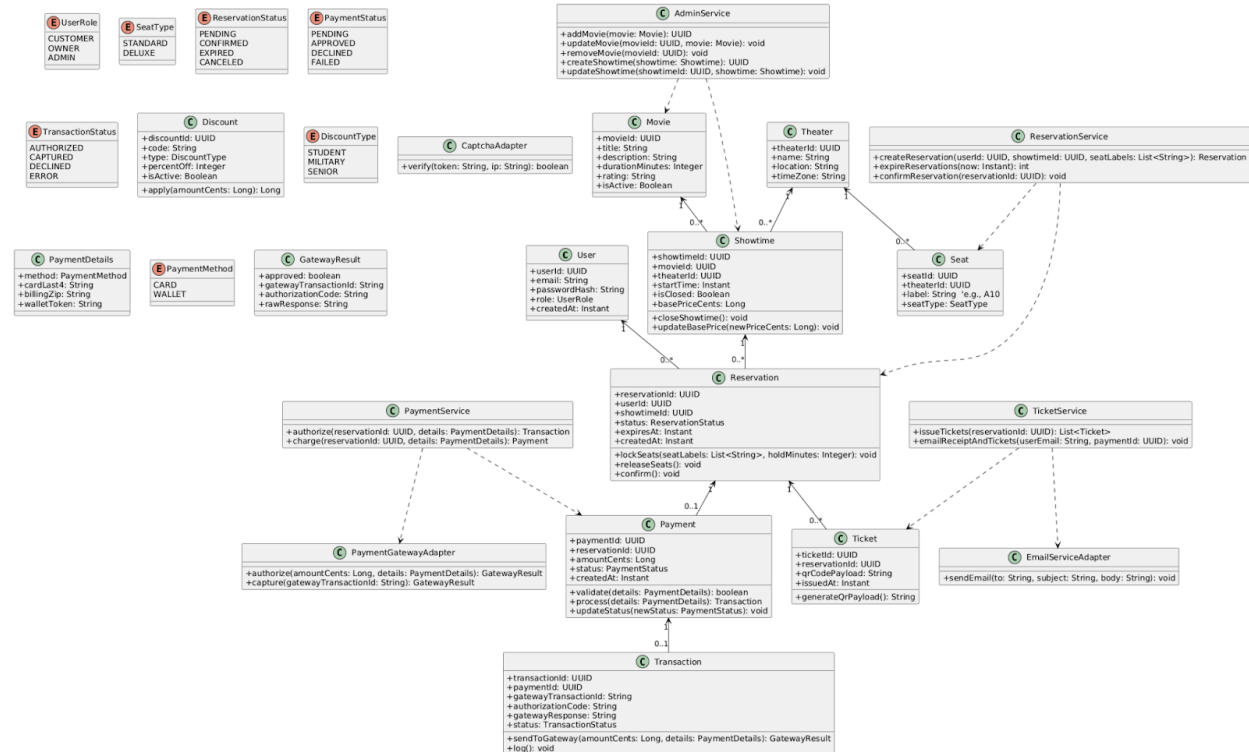
6.3 UML Class Diagram (Design-Level)

Theater Ticketing System

Below is a **design-level** UML definition you can render using PlantUML (recommended), or you can manually draw it with the same classes + relationships.

6.3.1 PlantUML

Theatre Ticketing System UML Diagram



This class diagram represents a movie theater reservation system where users (customers, owners, and admins) can browse movies, reserve seats for specific showtimes, make payments, and receive tickets. Movies are shown at theaters through scheduled showtimes, and each theater contains multiple seats with different types. A user creates a reservation for seats at a showtime, then makes a payment that is processed through a payment gateway. If the payment is successful, tickets are generated and can be emailed to the user. The system also includes services that handle reservations, payments, ticket issuing, and administrative tasks like adding movies or creating showtimes.

6.4 Class Specifications (Concise but “Rubric-Level” Detailed)

6.4.1 Reservation

Attributes:

- reservationId: UUID

Theater Ticketing System

- userId: UUID
- showtimeId: UUID
- status: ReservationStatus
- expiresAt: Instant

Operations:

- lockSeats(seatLabels: List<String>, holdMinutes: Integer): void
- releaseSeats(): void
- confirm(): void

6.4.2 Payment / Transaction

Purpose: Processes and records checkout payments.

Payment Attributes: paymentId: UUID, reservationId: UUID, amountCents: Long, status: PaymentStatus, createdAt: Instant

Payment Operations:

- validate(details: PaymentDetails): boolean
- process(details: PaymentDetails): Transaction
- updateStatus(newStatus: PaymentStatus): void

Transaction Attributes: transactionId: UUID, paymentId: UUID, gatewayTransactionId: String, authorizationCode: String, gatewayResponse: String, status: TransactionStatus

Transaction Operations:

- sendToGateway(amountCents: Long, details: PaymentDetails): GatewayResult
- log(): void

6.4.3 Ticket

Purpose: Represents issued entry tickets tied to a reservation.

Attributes: ticketId: UUID, reservationId: UUID, qrCodePayload: String, issuedAt: Instant

Operations:

Theater Ticketing System

- generateQrPayload(): String

6.4.4 AdminService

Purpose: Admin management of movies/showtimes/pricing.

Operations:

- addMovie(movie: Movie): UUID
- updateMovie(movieId: UUID, movie: Movie): void
- removeMovie(movieId: UUID): void
- createShowtime(showtime: Showtime): UUID
- updateShowtime(showtimeId: UUID, showtime: Showtime): void

6.5 Development Plan and Timeline

6.5.1 Task Partitioning (Work Packages)

Task 1 - Project Setup and CI

- Repository setup
- Branching strategy
- Linting configuration
- Basic CI pipeline setup

Task 2 - Database Schema and ORM

Tables to be created:

- Users
- Movies
- Theaters
- Showtimes
- Seats

Theater Ticketing System

- Reservations
- Payments
- Transactions
- Tickets
- Discounts

Constraints:

- Seat uniqueness per showtime
- Reservation expiration handling

Task 3 - Reservation and Seat Locking

- Seat availability checks
- 10-minute seat hold timer
- Conflict handling for simultaneous reservations

Task 4 - Checkout and Payment Integration

- Payment authorization and processing
- Transaction recording
- Error handling and retry logic
- CAPTCHA verification during checkout

Task 5 - Ticket Generation and Email

- Ticket record creation
- QR code payload generation
- Email confirmation and digital receipt

Task 6 - Administrative Features

Theater Ticketing System

- Manage movies, showtimes, pricing, and theater layouts
- Role-based access control for administrative users

Task 7 - Testing and System Hardening

- Unit testing for services
- Integration testing for reservation and payment flows
- Security checks and system logging

6.5.2 Team Member Responsibilities

Alvin Hoang

- Payment integration development
- Implementation of PaymentService and GatewayAdapter
- Payment and transaction database logic
- Security checks related to payment processing

Brandon Tse

- Reservation and seat locking APIs
- Customer interface for seat selection and checkout
- CAPTCHA integration for checkout verification

David Nguyen

- Database schema design and constraints
- Ticket generation and email integration
- Backend administrative services and API endpoints

Each team member will contribute multiple commits to the repository within their assigned areas.

6.4.5 User

Theater Ticketing System

Purpose: Represents a user account for system access and role-based permissions (Customer, Owner, Admin).

Attributes:

- `userId`: UUID -Unique identifier for the user
- `email`: String -Login identifier and contact email
- `passwordHash`: String -Secure hashed password (never store raw passwords)
- `role`: UserRole -Access level (CUSTOMER, OWNER, ADMIN)
- `createdAt`: Instant -Account creation timestamp

Operations:

- *(Authentication is handled in the API/controller layer; the User entity stores identity and role data.)*

6.4.6 Movie

Purpose: Stores metadata for a movie available for ticket purchase.

Attributes:

- `movieId`: UUID -Unique movie identifier
- `title`: String -Movie name
- `description`: String -Summary/description text
- `durationMinutes`: Integer -Runtime in minutes
- `rating`: String -Content rating (e.g., PG-13)
- `isActive`: Boolean- Indicates if movie is currently listed

Operations:

- *(CRUD operations are performed via AdminService using `addMovie`, `updateMovie`, `removeMovie`.)*

6.4.7 Theater

Theater Ticketing System

Purpose: Represents a physical theater location within the system (San Diego area).

Attributes:

- theaterId: UUID -Unique theater identifier
- name: String -Theater name
- location: String -City/address description
- timeZone: String -Time zone identifier (e.g., America/Los_Angeles)

Operations:

- *(Primarily data storage; configuration managed by Admin tools.)*

6.4.8 Showtime

Purpose: Represents a scheduled screening of a movie at a specific theater and time.

Attributes:

- showtimeId: UUID -Unique showtime identifier
- movieId: UUID -Linked movie
- theaterId: UUID -Linked theater
- startTime: Instant -Start time of showing
- isClosed: Boolean -If showtime is closed for purchase
- basePriceCents: Long -Base ticket price in cents

Operations:

- closeShowtime(): void -Disables ticket purchases for this showtime
- updateBasePrice(newPriceCents: Long): void -Updates base price for pricing changes

6.4.9 Seat

Purpose: Represents a seat inside a theater. Seat availability depends on showtime and reservation status.

Attributes:

Theater Ticketing System

- seatId: UUID -Unique seat identifier
- theaterId: UUID -Theater to which the seat belongs
- label: String -Seat label (e.g., A10)
- seatType: SeatType -STANDARD or DELUXE

Operations:

- *(Seat locking/sale is controlled through ReservationService and reservation state transitions.)*

6.4.10 Discount

Purpose: Stores and applies discount policies (student, military, senior).

Attributes:

- discountId: UUID - Unique discount identifier
- code: String - Discount code or category label
- type: DiscountType - STUDENT, MILITARY, SENIOR
- percentOff: Integer - Percentage discount applied (0-100)
- isActive: Boolean - Whether the discount is enabled

Operations:

- apply(amountCents: Long): Long -Returns the discounted amount in cents

6.4.11 PaymentGatewayAdapter

Purpose: Encapsulates communication with a third-party payment provider (Stripe/PayPal) so the system can swap gateways without rewriting business logic.

Operations:

- authorize(amountCents: Long, details: PaymentDetails): GatewayResult -Requests authorization for payment

Theater Ticketing System

- capture(gatewayTransactionId: String): GatewayResult -Captures/settles an authorized payment

6.4.12 EmailServiceAdapter

Purpose: Sends receipts and ticket confirmations after successful purchase.

Operations:

- sendEmail(to: String, subject: String, body: String): void

6.4.13 CaptchaAdapter

Purpose: Validates CAPTCHA tokens during checkout to reduce bot abuse.

Operations:

- verify(token: String, ip: String): boolean

6.4.14 PaymentDetails

Purpose: Standardizes payment input data used by payment processing functions without storing sensitive card data.

Attributes:

- method: PaymentMethod -CARD or WALLET
- cardLast4: String -Last 4 digits for display/logging (no full card storage)
- billingZip: String -Billing ZIP validation support
- walletToken: String -Token for digital wallet payment (if used)

Operations:

- *(Data transfer object; used as input parameter.)*

6.4.15 GatewayResult

Purpose: Standardizes payment gateway responses for application logic and logging.

Attributes:

- approved: boolean -Whether payment was approved

Theater Ticketing System

- gatewayTransactionId: String -Gateway transaction reference
- authorizationCode: String -Authorization code (if approved)
- rawResponse: String -Raw gateway response for audit/debugging

Operations:

- *(Return object; used as output from gateway calls.)*

6.4.16 ReservationService (Design Notes)

Purpose: Handles business logic for reservation creation, seat locking, expiration, and confirmation.

Operations:

- createReservation(userId: UUID, showtimeId: UUID, seatLabels: List<String>): Reservation
- expireReservations(now: Instant): int
- confirmReservation(reservationId: UUID): void

6.4.17 PaymentService

Purpose: Handles payment authorization/charge and updates reservation/payment records accordingly.

Operations:

- authorize(reservationId: UUID, details: PaymentDetails): Transaction
- charge(reservationId: UUID, details: PaymentDetails): Payment

6.4.18 TicketService (Design Notes)

Purpose: Issues tickets (QR payload) after confirmed purchase and sends email confirmation.

Operations:

- issueTickets(reservationId: UUID): List<Ticket>
- emailReceiptAndTickets(userEmail: String, paymentId: UUID): void

6.5.3 Project Timeline (17 Week Schedule)

Weeks 1-3

- Project planning
- Requirements specification development
- Initial repository setup

Weeks 4-6

- Software design specification
- UML class diagram creation
- Software architecture planning

Weeks 7-8

- Database schema implementation
- Initial backend API setup

Weeks 9-11

- Reservation system development
- Seat locking and conflict resolution
- Seat selection user interface integration

Weeks 12-13

- Payment gateway integration
- Transaction logging and retry logic

Weeks 14-15

- Ticket generation system
- QR code payload generation

Theater Ticketing System

- Email confirmation service

Weeks 16-17

- Administrative features
- System testing and debugging
- Final documentation and project polish

7. Test Plan (Assignment 3)

This section defines the verification and validation test plan for the Theater Ticketing System designed in Assignment 2. The goal is to ensure that functional and non-functional requirements are satisfied, including correct payment processing, seat reservation safety, purchase restrictions, discount processing, administrative management, and protection against bot abuse. Tests are designed to be repeatable, traceable to requirements in Section 3, and mapped to the architecture and UML components in Section 6.

7.1 Test Strategy

The system will be tested using the following levels of testing:

- **Unit Testing:** Verify correctness of individual classes and service methods (e.g., `ReservationService`, `PaymentService`, `TicketService`, discount logic).
- **Integration Testing:** Verify correct interaction between services and dependencies (DB, Payment Gateway Adapter, Email Adapter, CAPTCHA Adapter).
- **End-to-End (E2E) Testing:** Verify complete user flows from UI/API entry points through services to persistence (reservation → checkout → payment → ticket/email).
- **Security/Abuse Testing:** Verify CAPTCHA enforcement, rate limiting/rapid attempts, input validation, and safe handling of payment failures.

7.2 Test Environment

Tests will be executed in a controlled environment to ensure repeatable results.

Hardware/Client

- Desktop/laptop and mobile devices for UI verification.

Theater Ticketing System

- Browsers: Chrome, Edge, Safari, Firefox.

Server

- Local/dev deployment of the REST API (Controllers + Services).

Database

- Relational database test instance populated with known test data:
 - Movies, showtimes, theaters, seat maps, user accounts, discount categories.

External Services

To keep tests consistent and not dependent on live third-party services:

- **PaymentGatewayAdapter** will be tested using a **mock/sandbox gateway** that can simulate:
 - Approved authorization
 - Declines
 - Timeouts/network failures
- **EmailServiceAdapter** will be tested using a **mock email service** (capture outgoing messages in logs or test inbox).
- **CaptchaAdapter** will be tested using a **test mode** where tokens can be marked valid/invalid.

7.3 Test Scope and Traceability

The following table shows what requirements are being tested and where those behaviors exist in the design.

Requirement	Feature	Primary Components Tested	Test Type
3.2.3	Seat Reservation	ReservationService, Reservation, DB seat-locking logic	Unit, Integration, E2E

Theater Ticketing System

3.2.1	Process Ticket Payment	PaymentService, Payment, Transaction, PaymentGatewayAdapter	Unit, Integration, E2E
3.2.2	Payment Authorization Validation	PaymentService, PaymentGatewayAdapter	Unit, Integration
3.2.4	Ticket Purchase Restrictions	API validation layer + ReservationService / checkout validation	Unit, Integration
3.2.5	Bot Prevention	CaptchaAdapter, checkout controller, rate-limit logic	Integration, Security
3.2.6	Discount Processing	Discount, checkout pricing logic	Unit, Integration
3.2.7	Administrator Management	AdminService, admin controllers, DB	Unit, Integration

7.4 Test Data (Test Sets / Vectors)

The following test sets will be used across multiple tests:

- **Users**
 - Customer accounts (valid credentials)
 - Owner/admin accounts (valid credentials)
 - Invalid login user (wrong password)
- **Showtimes**
 - Showtime in the future (valid purchases allowed)
 - Showtime within 10 minutes after start time (purchases should be disabled)
 - Showtime more than 14 days in the future (purchases should be disabled)
- **Seats**
 - Available seat(s): e.g., A10, A11

Theater Ticketing System

- Already reserved/locked seat(s): e.g., B05
- **Payment Results (Mock Gateway)**
 - Approved
 - Declined
 - Timeout/failure
- **Discounts**
 - Valid discount category: STUDENT (e.g., 10% off)
 - Invalid/disabled discount
- **Bot/Abuse Conditions**
 - Invalid CAPTCHA token
 - Rapid repeated checkout attempts from same IP/session

7.5 Detailed Test Plans (By Feature)

7.5.1 Seat Reservation (Requirement 3.2.3)

Target design components: ReservationService, Reservation, Seat, DB constraints/locking

Goal: Ensure seats cannot be double-booked and reservation holds expire correctly.

Tests

1. **Reserve available seat (basic success)**
 - Input: valid MovieID, ShowtimeID, seats [A10, A11]
 - Expected: reservation created with status PENDING; seats locked for 10 minutes; ReservationID returned.
2. **Reserve seat conflict (double-book prevention)**
 - Test vector: two users attempt seat A10 for same showtime within a short interval.

Theater Ticketing System

- Expected: first succeeds; second receives “seat unavailable” error; no duplicate locks created.

3. Reservation expiration releases seats

- Create reservation; wait or simulate time > 10 minutes.
- Expected: reservation becomes EXPIRED; seats become available again; database updated.

4. Invalid showtime or seat label

- Input: invalid ShowtimeID or seat label not in theater layout.
- Expected: error response; no seat locks created.

7.5.2 Payment Authorization Validation (Requirement 3.2.2)

Target design components: `PaymentService.authorize(...)`,
`PaymentGatewayAdapter.authorize(...)`, `Payment.validate(...)`

Goal: Ensure payments are validated and authorization responses are handled correctly.

Tests

1. Authorization approved

- Input: valid reservation + valid payment details
- Mock gateway: returns approved with authorization code
- Expected: authorization success; transaction record created with status AUTHORIZED.

2. Authorization declined

- Mock gateway: returns declined
- Expected: user receives decline message; transaction logged; no tickets issued.

3. Authorization gateway timeout/failure

- Mock gateway: network error/timeout

Theater Ticketing System

- Expected: system returns “try again” error; error logged; reservation remains PENDING.

4. Invalid payment input format

- Missing required fields (e.g., billing ZIP)
- Expected: validation fails before gateway call; clear error message returned.

7.5.3 Process Ticket Payment (Requirement 3.2.1)

Target design components: `PaymentService.charge(...)`,
`Payment.process(...)`, `Transaction`, DB write operations

Goal: Ensure a successful charge records payment/transaction and triggers ticket/email flow.

Tests

1. Successful payment flow

- Preconditions: reservation is valid and not expired
- Mock gateway: approved/captured
- Expected:
 - Payment status becomes APPROVED
 - Transaction status becomes CAPTURED
 - Reservation becomes CONFIRMED
 - Tickets issued and email sent

2. Payment declined

- Mock gateway: declined
- Expected:
 - Payment status DECLINED or FAILED
 - Reservation remains PENDING (until expiration)

Theater Ticketing System

- No ticket issuance
- Error displayed

3. Payment amount mismatch

- Input: altered amount not matching reservation total
- Expected: reject payment before gateway call; log mismatch attempt.

7.5.4 Ticket Purchase Restrictions (Requirement 3.2.4)

Target design components: checkout validation layer, `ReservationService`, Showtime rules

Goal: Ensure system enforces fairness and time-based restrictions.

Tests

1. Ticket limit enforcement (20 max)

- Attempt to purchase 21 tickets in one transaction
- Expected: restriction error message; no payment attempt.

2. 14-day advance purchase restriction

- Attempt to purchase tickets for showtime > 14 days away
- Expected: restriction error message; checkout disabled.

3. 10-minute-after-start restriction

- Attempt purchase after showtime has started + 10 minutes
- Expected: checkout blocked; showtime marked closed or purchase refused.

7.5.5 Bot Prevention (Requirement 3.2.5)

Target design components: `CaptchaAdapter.verify(...)`, checkout controller, abuse detection logic

Goal: Ensure automated abuse is reduced and suspicious activity is blocked.

Tests

Theater Ticketing System

1. CAPTCHA required at checkout

- Input: missing CAPTCHA token
- Expected: checkout rejected; user prompted to complete CAPTCHA.

2. Invalid CAPTCHA token

- Mock captcha: invalid result
- Expected: checkout blocked; error logged.

3. Rapid repeated purchase attempts

- Simulate many checkout attempts within short time from same IP/session
- Expected: temporary block triggered; response indicates suspicious activity.

7.5.6 Discount Processing (Requirement 3.2.6)

Target design components: `Discount.apply( . . . )`, pricing logic during checkout

Goal: Ensure discount logic applies correctly and is logged.

Tests

1. Valid discount applied

- Input: discount category STUDENT
- Expected: discounted amount computed correctly; discount usage logged.

2. Invalid or disabled discount

- Input: discount not active or invalid
- Expected: no discount applied; error message or ignore with log entry.

3. Discount edge cases

- 0% off and 100% off (if allowed)
- Expected: correct final amount computed, no negative totals.

7.5.7 Administrator Management (Requirement 3.2.7)

Theater Ticketing System

Target design components: AdminService, admin controllers, DB writes

Goal: Ensure admins can manage movies/showtimes and unauthorized users cannot.

Tests

1. Admin creates showtime

- Input: valid movie + theater + time
- Expected: showtime created; visible to customers.

2. Non-admin access blocked

- Customer attempts admin action
- Expected: access denied (403/unauthorized).

3. Admin updates pricing

- Update base price for a showtime
- Expected: DB updated; new price appears in checkout totals.

7.6 Failure Modes Covered

This plan covers the following failure categories:

- Seat conflicts/double booking
- Reservation expiration timing issues
- Payment declines and gateway failures
- Invalid inputs and amount tampering attempts
- Purchase restriction violations (time + quantity)
- Bot abuse via repeated attempts or missing/invalid CAPTCHA
- Unauthorized access to admin functionality
- External dependency failures (email/payment downtime) using mocks

7.7 Pass/Fail Criteria

A test run is considered **passing** if:

- All **critical path** tests pass:
 - Seat reservation success + conflict protection
 - Payment approval path
 - Ticket issuance + email confirmation after approved payment
 - Restriction enforcement (ticket limit + time rules)
 - CAPTCHA enforcement at checkout
- No high-severity security tests fail (e.g., bypassing CAPTCHA or admin access control).
- All failed test cases are reproducible, logged, and mapped to a known defect with priority assigned.

7.8 Test Case Documentation (Excel)

All test cases in this plan will be mirrored in the required Excel test case spreadsheet, using unique test case IDs and mapping each case to the requirement IDs in Section 3 (e.g., 3.2.1, 3.2.3, 3.2.5). The spreadsheet will include the test steps, test data, expected results, actual results, and pass/fail status.